```
        }
    }
```

Κώδικας 5.11 *Παράδειγμα while με break*

Καταρχάς στον κώδικα 5.11 δηλώνουμε και αρχικοποιούμε τη μεταβλητή i στο 0. Παράλληλα δηλώνουμε τη μεταβλητή rI. Στη συνέχεια δημιουργούμε το αντικείμενο generator. Πρόκειται για μια γεννήτρια τυχαίων αριθμών. Σημειώστε πως για να χρησιμοποιήσετε ένα τέτοιο αντικείμενο θα πρέπει να εισάγετε στο πρόγραμμά σας τη βιβλιοθήκη Java.util.Random. Αυτό επιτυγχάνεται αν εισάγετε αμέσως μετά τη δήλωση του πακέτου τη σειρά import Java.util.Random; Στη συνέχεια αναπτύσσουμε μια επαναληπτική διαδικασία while. Η συνθήκη ελέγχου είναι i<10, δηλαδή η επαναληπτική διαδικασία θα επαναλαμβάνεται όσο το i θα είναι μικρότερο από το 10. Δεδομένου ότι το i αρχικοποιήθηκε στο 0 και σε κάθε επανάληψη αυξάνεται κατά 1, θα εκτελεστούν 10 το πολύ επαναλήψεις. Αυτό είναι συμβατό με την απαίτηση να τυπώσουμε 10 το πολύ τυχαίους ακέραιους. Στη συνέχεια χρησιμοποιούμε τον generator για να παράξουμε έναν τυχαίο αριθμό. Η συνάρτηση nextInt παράγει έναν τυχαίο ακέραιο από το 0 έως την τιμή του ορίσματός της μείον 1, δηλαδή από το 0 έως το 9. Σε αυτόν τον αριθμό προσθέτουμε το 1, οπότε ο ακέραιος που εκχωρείται στη μεταβλητή rI είναι ένας τυχαίος από το 1 έως το 10. Μετά ελέγχουμε την τιμή της rI και αν είναι μεγαλύτερη ή ίση με το 2, απλώς την τυπώνουμε διαφορετικά, ισχύει η απαίτηση σύμφωνα με την οποία αν κάποια από τις τυχαίες τιμές είναι μικρότερη από το 2, τότε θα πρέπει να την τυπώσουμε και να σταματήσουμε περαιτέρω εκτύπωση τυχαίων αριθμών. Αυτό ακριβώς κάνει ο κώδικας 5.11. Στη συνέχεια τυπώνουμε την τιμή που είναι μικρότερη από το 2 και αμέσως μετά εκτελείται η break που διακόπτει την επαναληπτική διαδικασία και μεταφέρει τη ροή έξω από την επαναληπτική διαδικασία όπου ακολουθεί και η έξοδος της συνάρτησης prtRandoms.
Δύο σχόλια σχετικά με τον κώδικα 5.11. Γενικά είναι καλό στα προγράμματά μας να αποφεύγουμε τη φλυαρία, δηλαδή όπου είναι εφικτό να προτιμούμε τον βραχύτερο έναντι του μακρύτερου κώδικα. Προσέξτε πως ο κώδικας System.out.println(rI) εκτελείται και στις δύο διακλαδώσεις της if, δηλαδή είτε ισχύει η συνθήκη rI>=2 είτε όχι. Ένας τέτοιος κώδικας μπορεί να βγει πάντα έξω από την if έτσι ώστε μια παρουσία του να είναι αρκετή.

Ένα άλλο θέμα που θα μπορούσαμε να βελτιώσουμε σχετίζεται με την αύξηση του i. Αφού ελέγξουμε το i στην επόμενη γραμμή, το αυξάνουμε κατά 1. Το ίδιο ακριβώς αποτέλεσμα μπορούμε να πετύχουμε αν αυξήσουμε το i στη συνθήκη ελέγχου με χρήση του μεταθεματικού τελεστή προσαύξησης. Προσοχή όμως! Αν η αύξηση γίνει όπως γίνεται στον κώδικα 5.11, δεν έχει ιδιαίτερη σημασία αν χρησιμοποιηθεί ο προθεματικός ή ο μεταθεματικός τελεστής προσαύξησης. Αν όμως η αύξηση γίνει μέσα στη συνθήκη, τότε η χρήση του μεταθεματικού τελεστή διαφοροποιεί ουσιαστικά τη συμπεριφορά του κώδικα. Μελετήστε πώς την αλλάζει και γιατί. Ο κώδικας 5.12 παράγει το ίδιο ακριβώς αποτέλεσμα με τον κώδικα 5.11 με το πλεονέκτημα ότι είναι βραχύτερος.

```
static void prtRandoms() { //Κώδικας 5.12
    int i = 0, rI;
    Random generator = new Random();
    while (i++ < 10) {
        rI = generator.nextInt(10) + 1;
        System.out.println(rI);
        if (rI < 2) {
            break;
        }
    }
}
```

Κώδικας 5.12 *Βραχύτερη έκδοση της συνάρτησης prtRandoms*

Ας υποθέσουμε τώρα πως θέλουμε να τυπώσουμε 10 τυχαίες ακέραιες τιμές στην κλίμακα 1 έως 10 εξαιρώντας τις τιμές 3 και 4, δηλαδή 10 τιμές που ανήκουν στο σύνολο {1..10}-{3,4}. Η λέξη-κλειδί continue θα μας βοηθήσει στην επίλυση αυτού του προβλήματος.

```
static void prt10Randoms() { //Κώδικας 5.13
    int i = 0, rI;
    Random generator = new Random();
    while (i < 10) {
        rI = generator.nextInt(10) + 1;
        if (rI == 3 || rI == 4) {
            continue;
        }
        System.out.println(rI);
        i++;
    }
}
```

Κώδικας 5.13 *Παράδειγμα while με χρήση της λέξης-κλειδί continue*

Μέσα στον βρόχο while του κώδικα 5.13 παράγουμε καταρχάς έναν τυχαίο αριθμό από 1 έως 10. Στη συνέχεια ελέγχουμε αν πρόκειται για αριθμό που εξαιρείται. Αν όντως πρόκειται για αριθμό που εξαιρείται, καλούμε την continue ώστε να μεταφέρουμε τη ροή στην αρχή του βρόχου όπου ελέγχεται η συνθήκη i<10. Αν δεν πρόκειται για αριθμό που εξαιρείται, προχωρούμε κανονικά στην εκτύπωσή του και στη συνέχεια αυξάνουμε τη μεταβλητή ελέγχου i κατά 1. Προσοχή, εδώ η i θα ήταν λάθος να αυξηθεί μέσα στη συνθήκη ελέγχου. Κάτι τέτοιο θα είχε ως αποτέλεσμα την παραγωγή συνολικά 10 ακέραιων και την εκτύπωση μόνο όσον είναι διάφοροι από το 3 και το 4, δηλαδή να εκτυπωθούν πιθανότατα λιγότεροι από 10 ακέραιους που προσδιορίζουν οι απαιτήσεις του προβλήματός μας.

5.2.2 Η επαναληπτική δομή do-while

Η γενική μορφή της do-while έχει ως εξής:

```
do {
    statement(s);
} while (condition);
```

Η διαφορά της από την while είναι πως η συνθήκη ελέγχεται στο τέλος. Επομένως, η do-while εγγυάται πως ο κώδικας μέσα στο σώμα της θα εκτελεστεί τουλάχιστον μία φορά.

Ας υποθέσουμε πως θέλουμε να λάβουμε μια σειρά από ακέραιους από τον χρήστη και να υπολογίσουμε το άθροισμά τους και επιπλέον, να σταματήσουμε τη διαδικασία εισόδου από τον χρήστη όταν αυτός δώσει την τιμή 0.

Σε αυτό το πρόβλημα, ο βρόχος do-while θα μας φανεί πιο χρήσιμος από τον βρόχο while, καθώς είναι αναγκαίο να λάβουμε μια τουλάχιστον είσοδο από τον χρήστη.

```
static void calcSum() { //Κώδικας 5.14
    int i;
    int sum = 0;
    Scanner in = new Scanner(System.in);
    do {
        System.out.print("Enter int (0 terminates input):");
        i = in.nextInt();
        sum += i;
    } while (i != 0);
    System.out.println("Sum = " + sum);
}
```

Κώδικας 5.14 *Παράδειγμα χρήσης του βρόχου do-while*

Στον κώδικα 5.14 ορίζουμε τη μεταβλητή i που θα χρησιμοποιήσουμε για να καταχωρούμε την είσοδο του χρήστη. Τη μεταβλητή i δεν είναι αναγκαίο να την αρχικοποιήσουμε, καθώς είναι βέβαιο πως πριν χρησιμοποιηθεί θα εκχωρηθεί τιμή σε αυτήν. Στη συνέχεια ορίζουμε τη μεταβλητή sum. Είναι αναγκαίο να αρχικοποιήσουμε την sum στο 0, καθώς αυτή προσαυξάνεται κατά την είσοδο του χρήστη σε κάθε βήμα της επαναληπτικής διαδικασίας do-while. Ακολούθως ορίζουμε ένα αντικείμενο της κλάσης Scanner για να το χρησιμοποιήσουμε για είσοδο από τον χρήστη. Στο επόμενο βήμα μπαίνουμε στην επαναληπτική διαδικασία do-while όπου ζητάμε και παίρνουμε έναν ακέραιο από τον χρήστη με τον οποίο ενημερώνουμε τον αθροιστή sum. Στο τέλος της do-while γίνεται ο έλεγχος της συνθήκης. Αν ο χρήστης έδωσε ακέραιο διάφορο του 0, η διαδικασία επαναλαμβάνεται αλλιώς τερματίζεται.

Όλες οι επαναληπτικές διαδικασίες μπορούν να υλοποιηθούν με χρήση οποιασδήποτε επαναληπτικής δομής. Ωστόσο, κάποιες δομές παρέχουν διευκολύνσεις ανάλογα με τις απαιτήσεις που πρέπει να υλοποιηθούν. Στον κώδικα 5.15 επιλύουμε το ίδιο πρόβλημα με αυτό του κώδικα 5.14 αλλά χρησιμοποιώντας τη δομή while αντί της δομής do-while.

```
static void calcSum() { //Κώδικας 5.15
    int i;
    int sum = 0;
    Scanner in = new Scanner(System.in);
    System.out.print("Enter int (0 terminates input):");
    i = in.nextInt();
    while (i != 0) {
        sum += i;
        System.out.print("Enter int (0 terminates input):");
        i = in.nextInt();
    }
    System.out.println("Sum = " + sum);
}
```

Κώδικας 5.15 *Υλοποίηση της calcSum με while αντί do-while*

Προσέξτε πως στον κώδικα 5.15 οι εντολές εισόδου από τον χρήστη εμφανίζονται δύο φορές, μία έξω από τον βρόχο και μια μέσα στον βρόχο μετά την επεξεργασία που στην περίπτωσή μας είναι η πρόσθεση του ακέραιου i στον αθροιστή sum. Πρόκειται για μια τυπική μεταφορά ενός do-while σε while. Ωστόσο, η διπλή παρουσία των εντολών εισόδου μπορεί να αποφευχθεί αν οργανώσουμε τον κώδικά μας όπως φαίνεται στον κώδικα 5.16.

```
static void calcSum() { //Κώδικας 5.16
    int i;
    int sum = 0;
    Scanner in = new Scanner(System.in);
    while (true) {
        System.out.print("Enter int (0 terminates input):");
        i = in.nextInt();
        if (i == 0) {
            break;
        }
        sum += i;
    }
    System.out.println("Sum = " + sum);
}
```

Κώδικας 5.16 *Προσομοίωση της do-while με while*

Στον κώδικα 5.16 προσομοιώνουμε τη λειτουργία της do-while με χρήση της while. Η ροή του κώδικα θα εισέλθει οπωσδήποτε μια τουλάχιστον φορά στον βρόχο while, καθώς η συνθήκη ελέγχου είναι πάντα true.

Στην ουσία ο έλεγχος του βρόχου γίνεται από την if, η οποία αν εντοπίσει τιμή ίση με το 0, καλεί την break που προκαλεί έξοδο από τον βρόχο.
Συχνά δημιουργείται η αντίληψη ότι η continue κατευθύνει τη ροή του προγράμματος στην αρχή της επαναληπτικής διαδικασίας. Η αντίληψη αυτή όμως είναι λανθασμένη. Η continue κατευθύνει τη ροή στη συνθήκη ελέγχου του βρόχου. Αν ο βρόχος είναι τύπου while, τότε η αρχή της διαδικασίας συμπίπτει με τη συνθήκη ελέγχου. Αν όμως ο βρόχος είναι τύπου do-while, τότε η συνθήκη βρίσκεται στο τέλος. Ο κώδικας 5.17 επιδεικνύει τη λειτουργία της continue σε έναν do-while βρόχο.

```
static void checkContinue() {
    int i = 0;
    do {
        i++;
        System.out.println(i);
        if (i == 1) {
            continue;
        }
        System.out.println(i);
    } while (i < 1);
}
```

Κώδικας 5.17 *Η λειτουργία της continue*

Ο κώδικας 5.17 όταν εκτελείται τυπώνει 1. Το αποτέλεσμα του κώδικα είναι απόδειξη ότι η continue μεταφέρει τη ροή στη συνθήκη ελέγχου και όχι απλώς στην αρχή του βρόχου. Με την είσοδο μέσα στον βρόχο, η μεταβλητή i αυξάνεται κατά 1 και γίνεται 1 από 0 που ήταν η τιμή στην οποία αρχικοποιήθηκε στην πρώτη γραμμή της checkContinue. Στη συνέχεια τυπώνεται η τιμή της i, δηλαδή το 1. Ακολούθως ελέγχεται η τιμή της i, η συνθήκη i==1 βρίσκεται αληθής και εκτελείται το continue. Αν το continue κατηύθυνε τη ροή στην αρχή του βρόχου, τότε το i θα γινόταν 2 και στη συνέχεια θα τυπωνόταν πριν φτάσει στη συνθήκη ελέγχου και εξέλθει από τον βρόχο. Όμως, η continue κατευθύνει άμεσα τη ροή στη συνθήκη ελέγχου η οποία φυσικά για την τιμή του i ίση με το 1 είναι ψευδής και έτσι ο βρόχος διακόπτεται έχοντας εκτυπώσει μόνο το 1.

5.2.3 Η επαναληπτική δομή for

Η for παρουσιάζει μεγάλη ευελιξία και είναι ίσως η συχνότερα χρησιμοποιούμενη επαναληπτική δομή. Η γενική μορφή της έχει ως εξής:

```
for (initialization; condition; statement) {
     statement(s);
}
```

Η for επομένως αποτελείται από τέσσερα μέρη. Το initialization είναι το τμήμα εκείνο στο οποίο τυπικά αρχικοποιείται ή και δηλώνεται η μεταβλητή ελέγχου του βρόχου. Η condition είναι η συνθήκη που ελέγχει αν θα συνεχιστεί η επαναληπτική διαδικασία ή όχι. Το statement μπορεί να είναι μια οποιαδήποτε εντολή ή συνάρτηση. Στην τυπική περίπτωση, στο statement μεταβάλλεται η τιμή της μεταβλητής ελέγχου. Το τέταρτο μέρος είναι το μπλοκ της for στο οποίο μπορούμε να τοποθετήσουμε ένα ή περισσότερα statements.

Το initialization εκτελείται μια φορά κατά την είσοδο στον βρόχο. Στη συνέχεια εκτελείται η condition. Αν βρεθεί αληθής, η ροή κατευθύνεται στο μπλοκ της for. Αφού εκτελεστούν οι statements του μπλοκ, η ροή κατευθύνεται στο statement εντός του κύριου σώματος της for. Μετά την εκτέλεση του statement εντός του κύριου σώματος, η ροή κατευθύνεται στην condition για να ακολουθήσει μια κυκλική διαδρομή του τύπου condition-statement(s)-statement έως ότου βέβαια η condition λάβει την τιμή false. Σε αυτήν την περίπτωση, διακόπτεται η for και η ροή του προγράμματος κατευθύνεται στην έξοδο του βρόχου. Ας δούμε καταρχάς μια απλή εφαρμογή της for. Ο κώδικας 5.18, τυπώνει τους ακεραίους από 0 έως 9.

```
for (int i = 0; i < 10; i++) {
     System.out.println(i);
}
```

Κώδικας 5.18 *Εκτύπωση των ακέραιων 0..9 με την for*

Η πρώτη τιμή που εκτυπώνεται είναι η τιμή 0. Αυτό δείχνει πως η ροή εκτέλεσης πρώτα εισέρχεται στο μπλοκ της for και στη συνέχεια εκτελεί την προσαύξηση του i στο κυρίως σώμα της for. Η δε τελευταία τιμή είναι το 9. Πράγματι, μόλις τυπωθεί το 9, η ροή κατευθύνεται στην πρόταση προσαύξησης του i που γίνεται 10. Στη συνέχεια, η συνθήκη ελέγχου καθίσταται ψευδής και οδηγεί στην έξοδο του βρόχου.
Ας σημειωθεί πως στο τμήμα αρχικοποίησης της for δεν είναι υποχρεωτικό να δηλώνεται η μεταβλητή ελέγχου. Ωστόσο, αποτελεί καλή πρακτική η δήλωσή της εφόσον αυτή δεν είναι χρήσιμη έξω από την for. Κάτι ακόμη που πρέπει να αποσαφηνίσουμε είναι πως κανένα από τα τέσσερα τμήματα της for δεν είναι υποχρεωτικό.

Στη συνέχεια δίνουμε παραδείγματα χρήσης της for μέσα από τα οποία γίνονται σαφείς οι παρατηρήσεις μας σε σχέση με αυτήν. Πιο συγκεκριμένα, υλοποιούμε με for κώδικες που αξιοποιούν την while ή την do-while και έχουν ήδη παρουσιαστεί μέχρι εδώ σε αυτήν την ενότητα. Με αυτόν τον τρόπο θα αποκτήσουμε πληρέστερη εικόνα της σχετικής ευελιξίας της for.
Ο κώδικας 5.10 που επιδεικνύει τη χρήση της while, υλοποιείται με for όπως δείχνει ο κώδικας 5.19.

```
static void forDemo() { //Κώδικας 5.19
    int i;
    for (i = 10; i > 0; i--) {
        System.out.print("i>0 ");
    }
    System.out.println("i=" + i);
}
```

Κώδικας 5.19 *Ίδιο αποτέλεσμα με την while του κώδικα 5.10*

Στον κώδικα 5.19 η μεταβλητή ελέγχου i δεν δηλώνεται στο τμήμα αρχικοποίησης της for. Αυτό είναι αναγκαίο, καθώς θέλουμε να προσπελάσουμε τη μεταβλητή έξω από τα όρια της for. Αν την είχαμε δηλώσει στο τμήμα αρχικοποίησης της for, η εμβέλεια της μεταβλητής θα περιοριζόταν στα τμήματα της for.
Η συνάρτηση prtRandoms που παρουσιάζεται στον κώδικα 5.12 μεταγράφεται με for όπως φαίνεται στον κώδικα 5.20:

```
static void prtRandoms() { //Κώδικας 5.20
    Random generator = new Random();
    int rI;
    for (int i = 0; i++ < 10;) {
        rI = generator.nextInt(10) + 1;
        System.out.println(rI);
        if (rI < 2) {
            break;
        }
    }
}
```

Κώδικας 5.20 *Η prtRandoms του κώδικα 5.12 υλοποιημένη με for*

Προσέξτε πως σε αυτήν την περίπτωση το τρίτο τμήμα της for δεν χρησιμοποιήθηκε. Όπως αναφέραμε προηγουμένως, όλα τα τμήματα της for είναι προαιρετικά.
Η συνάρτηση prtRandoms που παρουσιάστηκε στον κώδικα 5.13 μεταγράφεται με for όπως δείχνει ο κώδικας 5.21:

```
static void prtRandoms() { //Κώδικας 5.21
    int rI;
    Random generator = new Random();
    for (int i = 0; i < 10; i++) {
        rI = generator.nextInt(10) + 1;
        if (rI == 3 || rI == 4) {
            continue;
        }
        System.out.println(rI);
    }
}
```

Κώδικας 5.21 *Η prtRandoms του κώδικα 5.13 υλοποιημένη με for*

Ο κώδικας 5.22 υλοποιεί την calcSum του κώδικα 5.14 χρησιμοποιώντας for αντί για do-while:

```
static void calcSum() { //Κώδικας 5.22
    int sum = 0;
    Scanner input = new Scanner(System.in);
    for (int i = 1; i != 0;) {
        System.out.print("Enter int (0 terminates input):");
        i = input.nextInt();
        sum += i;
    }
    System.out.println("Sum = " + sum);
}
```

Κώδικας 5.22 *Η calcSum του κώδικα 5.14 υλοποιημένη με for*

Ο κώδικας 5.23 παρέχει μια κάπως διαφορετική υλοποίηση του κώδικα 5.18:

```
 for (int i=0; i<10; System.out.println(i));
```

Κώδικας 5.23 *Εναλλακτική υλοποίηση του κώδικα 5.18*

Συχνά οι αρχάριοι προγραμματιστές έχουν την εντύπωση πως οι μεταβλητές ελέγχου της for είναι υποχρεωτικά τύπου int. Κάτι τέτοιο βέβαια δεν αληθεύει. Παρακάτω στον κώδικα 5.24, εκτυπώνουμε το αγγλικό αλφάβητο με κεφαλαίους χαρακτήρες:

```
static void prtAlpha() { //Κώδικας 5.24
    for (char c = 'A'; c <= 'Z'; c++) {
        System.out.print(c + " ");
    }
    System.out.println();
}
```

Κώδικας 5.24 *Εκτύπωση της Αγγλικής Αλφάβητου με for*

Η μεταβλητή που αρχικοποιείται στο τμήμα αρχικοποίησης της for δεν λαμβάνει υποχρεωτικά μέρος στη συνθήκη ελέγχου. Αυτό επιδεικνύεται στον κώδικα 5.25, ο οποίος τυπώνει διαδοχικούς ακέραιους αρχίζοντας από το 0 έως ότου ο χρήστης επιλέξει έξοδο.

```
static void prtInts() {//Κώδικας 5.25
    int v = 1;
    Scanner input = new Scanner(System.in);
    for (int i = 0; v > 0; i++) {
        System.out.println(i);
```

```
            System.out.print("Εισάγετε 0 για έξοδο:");
            v = input.nextInt();
        }
    }
```

Κώδικας 5.25 *Εκτύπωση διαδοχικών ακέραιων μέχρι να επιλέξει ο χρήστης έξοδο*

Όπως στις δομές επιλογής, έτσι και εδώ έχουμε τη δυνατότητα εμφωλευμένων επαναληπτικών δομών. Ο κώδικας 5.26 τυπώνει 10 φορές το αγγλικό αλφάβητο.

```
    static void nestedFor() { //Κώδικας 5.26
        for (int i = 0; i < 10; i++) {
            for (char c = 'A'; c <= 'Z'; c++) {
                System.out.print(c + " ");
            }
            System.out.println();
        }
    }
```

Κώδικας 5.26 *Παράδειγμα εμφωλευμένης for*

Ο κώδικας 5.27 έχει ως σκοπό να δώσει έμφαση στο γεγονός ότι τα διάφορα τμήματα της for είναι προαιρετικά. Τέτοιου είδους χρήση της for υποστηρίζεται μεν από τον μεταγλωττιστή, είναι ωστόσο ασυνήθιστη.

```
    static void emptyFor() { //Κώδικας 5.27
        int i = 0;
        for (;;) {
            System.out.println(i++);
            if (i > 10) {
                break;
            }
        }
    }
```

Κώδικας 5.27 *Παράδειγμα της for χωρίς κανένα από τα τρία κύρια τμήματα*

Σημείωση: Υπάρχει και μια επιπλέον μορφή της for, γνωστή ως ενισχυμένη (enhanced) for. Η ενισχυμένη for σχετίζεται με τις συλλογές, δηλαδή με δομές που είναι κατάλληλες για αποθήκευση πλήθους δεδομένων, π.χ. πίνακες. Η ενισχυμένη for παρουσιάζεται στην ενότητα 6.1.2.

5.3. Λυμένες Ασκήσεις

5.3.1 Εκτύπωση πραγματικών με δεκαδικό βήμα

Αναπτύξτε συνάρτηση που τυπώνει τους πραγματικούς αριθμούς από 1 έως 10 με βήμα 0.1.

Λύση
Μια απλή for με μεταβλητή ελέγχου τύπου double αρκεί για τη λύση αυτής της άσκησης.

```
    static void simpleForwithDouble() { //Κώδικας 5.28
        for (double i = 0; i <= 10; i += 0.1) {
            System.out.println(i);
        }
    }
```

Κώδικας 5.28 *Απλή for με μεταβλητή ελέγχου double και βήμα 0.1*

5.3.2 Εκτύπωση ακέραιων υπό συνθήκη

Να γράψετε πρόγραμμα που τυπώνει όλους τους ακέραιους από το 0..100 που διαιρούνται ακριβώς με το 4, το 6 και το 8.

Λύση
Η πρώτη σκέψη που μας έρχεται στο μυαλό είναι να ελέγξουμε κάθε ακέραιο από το 0 έως το 100 και αν διαιρείται ακριβώς με το 4, το 6 και το 8 να το εμφανίσουμε. Ωστόσο, υπάρχει και πιο αποδοτικός τρόπος. Αφού οι αριθμοί που θα τυπώνουμε διαιρούνται με το 4, το 6 και το 8, μπορούμε να ξεκινήσουμε από το 4 και να ελέγξουμε μόνο τα πολλαπλάσια του 4. Ελέγχουμε λοιπόν τα πολλαπλάσια του 4. Αν πληρούν και τις λοιπές προϋποθέσεις, τότε τα τυπώνουμε. Αν θεωρήσουμε πως και το 0 είναι πολλαπλάσιο του 4, τότε θα ξεκινήσουμε από το 0. Με αυτόν τον τρόπο, αντί να ελέγξουμε 100 περίπου ακέραιους, ελέγχουμε μόνο 25. Μπορούμε και εδώ να χρησιμοποιήσουμε μια for.

```
static void prtRealsConditional() { //Κώδικας 5.29
    for (int i = 0; i <= 100; i += 4) {
        if (i % 6 == 0 && i % 8 == 0) {
            System.out.println(i);
        }
    }
}
```

Κώδικας 5.29 *Εκτύπωση ακέραιων υπό συνθήκη*

5.3.3 Εκτύπωση και καταμέτρηση ακέραιων υπό συνθήκη

Να γράψετε πρόγραμμα που τυπώνει όσους ακεραίους στην περιοχή 0..1000 είναι πολλαπλάσια του 3 και ταυτοχρόνως τέλεια τετράγωνα, δηλαδή η τετραγωνική τους ρίζα είναι ακέραιος.
Μετράει και εκτυπώνει το πλήθος των ακέραιων που πληρούν τις περιγραφόμενες προϋποθέσεις.
Σημείωση: Η τετραγωνική ρίζα μιας αριθμητικής μεταβλητής i δίνεται από τη συνάρτηση sqrt της κλάσης Math : Math.sqrt(i).

Λύση
Το ερώτημα εδώ είναι ο έλεγχος για να διαπιστώσουμε αν η τετραγωνική ρίζα ενός αριθμού είναι ακέραιος. Καταρχάς την τετραγωνική ρίζα θα την υπολογίσουμε με τη συνάρτηση sqrt() της Math. Όπως είναι λογικό, η συνάρτηση αυτή επιστρέφει αριθμό τύπου double. Για να διαπιστώσουμε αν ο αριθμός αυτός είναι ακέραιος, θα ελέγξουμε αν το υπόλοιπο της διαίρεσης μεταξύ τετραγωνικής ρίζας και 1 είναι ίσο με το 0. Προκειμένου όμως να ελέγξουμε σωστά θα πρέπει να χρησιμοποιήσουμε προσεγγιστική ισότητα, όπως έχουμε επισημάνει στις ενότητες 3.2 και 4.1.4.1.

```
static boolean approximateEquals(double f1, double f2, double epsilon) {
    return Math.abs(f2 - f1) < epsilon;
}

static void search() { //Κώδικας 5.30
    int cnt = 0;
    for (int i = 0; i < 1000; i += 3) {
        if (approximateEquals(Math.sqrt(i) % 1, 0, 0.00001d)) {
            System.out.print(i + " ");
            cnt++;
        }
    }
    System.out.println();
```